

Supplemental Material: Providing Rapid Design Feedback for 3D Obstacle Course Games Using Constrained Solvability Queries

ZANDER MAJERCIK, Stanford University, USA and Roblox, USA

SHARON ZHANG, Stanford University, USA and Roblox, USA

WILLIAM WANG, Stanford University, USA

TEJAN KARMALI, Stanford University, USA

FANGJUN ZHOU, Stanford University, USA

YUCHENG YUAN, Stanford University, USA

JEAN-PEÏC CHOU, Stanford University, USA

MANEESH AGRAWALA, Stanford University, USA and Roblox, USA

KAYVON FATAHALIAN, Stanford University, USA and Roblox, USA

ACM Reference Format:

Zander Majercik, Sharon Zhang, William Wang, Tejan Karmali, Fangjun Zhou, Yucheng Yuan, Jean-Peïc Chou, Maneesh Agrawala, and Kayvon Fatahalian. 2026. Supplemental Material: Providing Rapid Design Feedback for 3D Obstacle Course Games Using Constrained Solvability Queries. *ACM Trans. Graph.* 44, 3, Article 1 (November 2026), 18 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Solve Rate comparisons

Fig. 1 compares solve rate for each of the methods compared with average solution time in Fig. 3 of the main paper. To compute the solve rate, we run each of our 56 queries over 5 seeds and consider the fraction of seeds solved in under 90s as the per-query solve rate. For each tertile, we then find the mean and inter-quartile range (top/bottom 25%-ile) over all the per-query solve rates (Fig. 1). Both Go-Explore (GE) and Go-Explore Policy-Selection (GE-PS) have a high solve rate ($> 90\%$) over both short and medium queries. Cell-Free Latent Go-Explore (CFLGE) has a much lower solve rate of 58% and 5% on short and medium buckets respectively. We observe the main advantages of the policy selection in the long regime where Go-Explore solves only 31% of queries compared to 51% solve by Go-Explore Policy-Selection. Policy-Based Go-Explore (PBGE) consistently could not solve any query. We suspect this is because the time it takes to train a policy to return to previous bins (and

Authors' Contact Information: Zander Majercik, majercik@stanford.edu, Stanford University, Stanford, California, USA and Roblox, San Mateo, California, USA; Sharon Zhang, szhang25@stanford.edu, Stanford University, Stanford, California, USA and Roblox, San Mateo, California, USA; William Wang, Stanford University, Stanford, California, USA; Tejan Karmali, Stanford University, Stanford, California, USA; Fangjun Zhou, Stanford University, Stanford, California, USA; Yucheng Yuan, Stanford University, Stanford, California, USA; Jean-Peïc Chou, Stanford University, Stanford, California, USA; Maneesh Agrawala, Stanford University, Stanford, California, USA and Roblox, San Mateo, California, USA; Kayvon Fatahalian, Stanford University, Stanford, California, USA and Roblox, San Mateo, California, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7368/2026/11-ART1

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

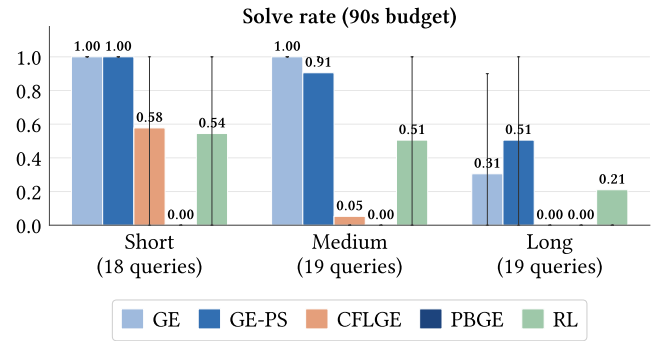


Fig. 1. Solve rate by Time to First Solution across the five methods: Go-Explore [Ecoffet et al. 2021] (GE), Go-Explore Policy-Selection (GE-PS), Cell-Free Latent Go-Explore [Gallouédec and Dellandréa 2023] (CFLGE), Policy-Based Go-Explore [Ecoffet et al. 2021] (PBGE), and Reinforcement Learning [Schulman et al. 2017] (RL) based with queries grouped into Short / Medium / Long buckets based on Go-Explore's mean solve time.

simulate the returning actions) is too costly for a 90s time budget. The pretrained RL policy separate from the Go-Explore algorithm has a 50% solve rate on short and medium queries and dips to 21% on long queries.

2 Policy-Selection Go-Explore

2.1 Background: The Go-Explore Algorithm (+ our modifications)

Go-Explore [Ecoffet et al. 2021] is a search-based framework designed for hard-exploration problems. We refer the reader to Extended Alg. 1 for full implementation of Go-Explore, along with our proposed modifications using the pretrained policy highlighted in red. Go-Explore addresses the limitations of reinforcement learning algorithms in exploration of the space by separating the search for new states from the act of returning to them, thereby avoiding detachment (losing track of exploration frontiers) and derailment (stochastic noise preventing returns to distant states). When enabled with parallel simulations, Go-Explore can return a solution

trajectory within a few seconds, which makes it suitable for our use case.

Core Exploration Loop The algorithm maintains an archive of “bins” representing different states. The exploration proceeds through a continuous loop:

- **Selection:** A bin is chosen from the archive according to a sampling heuristic.
- **Return:** The agent returns to the exact state of that bin without added exploration by resetting a deterministic simulator.
- **Explore:** The agent explores from that state for a fixed duration using random or intelligent actions.
- **Update:** New states are added to the archive, existing bins are updated if a better trajectory (higher score or shorter length) is discovered, and statistics related to the traversed bins are updated.

2.2 Binning and Bin Representation

To handle high-dimensional state spaces like pixel space observations, Go-Explore uses a binning function to map states into discrete bins. Given a level, we construct a 6-dimensional binning function based on: the agent’s XYZ position, the current timestep, the number of jumps the agent has remaining, and the position of a single interactable object (i.e. a block the agent can push). We allocate 32 bins each to the horizontal dimensions, and 8 to the vertical dimension. Allocation of bins to time, jumps remaining, and interactable object position is dependent on whether the relevant constraints are present in the level. See Alg. 1 below.

2.3 Scoring Function and Bin Sampling Criteria

To decide on the state stored per bin, we use the same scoring function as the original Go-Explore algorithm, which is to store the state within the bin at the end of a shortest trajectory. This scoring function prioritizes finding faster paths to the goal. Bins are selected from the archive using a weight inversely proportional to the number of times that bin has been seen during exploration (Lines 30–32 in Alg. 1).

2.4 Sampling for diverse solutions

When we find a trajectory, we clear the saved state of any bin through which the found trajectory traveled. This ensures both a) we never return two trajectories with exact sectional overlap, and b) we target resetting the search in areas affected by the found solution trajectory. This way other areas, dealing with in-progress trajectories, are less likely to be affected.

3 RL Pre-Training Details

This section described our procedure for training the agent used in our implementation of Go-Explore Policy-Selection.

3.1 Training Implementation

Our training process timesteps 8192 worlds in parallel. At the start of training, we first sample a set of 1024 obstacles uniformly at random from our obstacle DSL and upload them to GPU memory. The obstacle type (e.g. Lava Ring Drop, Lava Stairs) and parameter

values (e.g. platform length, lava width) are chosen uniformly at random (see Sec. 7). All 8192 worlds in parallel then sample from this set to begin a new episode. After 100 policy updates (where each update occurs after 40 simulation steps), we sample a new set of 1024 obstacles. We found that updating the 1024 obstacle set every 100 policy updates struck a good balance between ensuring a wide range of obstacles for training and avoiding the GPU-CPU sync incurred by updating the GPU memory buffer. We do not upload more than 1024 obstacles at a time to GPU memory because we wish to maximize the amount of GPU memory available for agent training. The agent policies are based on the actor-critic model, consisting of a fully connected Multi Layer Perceptron (MLP) with three hidden layers of width 256 and separate heads for the actor and critic and trained using PPO [Schulman et al. 2017].

3.2 Agent Observations and Reward Function

The agent observation space is as follows:

- **Raytraced Observations:** 128 rays traced in directions uniformly sampled on a sphere centered on the agent. These rays are traced to a maximum distance of 200m. Hits return distance to object and a one-hot encoding indicating whether the object is harmful. Missed rays are returned as hit distance 0.0 and label 0.
- **Object Observations:** The agent receives the dimensions, relative positions, and velocities (linear and angular) of up to 16 objects, sorted by the relative distance to the agent in the forward direction.
- **Agent Velocity:** XYZ vector representing the agent’s velocity.
- **Vector from agent to goal:** Agent-centric polar coordinate vector represent the direction and distance to the goal.
- **Closest distance:** a scalar representing the closest the agent has come to the goal in the current episode.
- **Previous action:** the agent’s previous taken action.

Agent actions are discrete 3-tuples $a \in \mathcal{A} : \{0, 1\} \times \{0, \dots, 7\} \times \{0, 1\}$, where the first component represents whether or not to move, the second component represents the agent relative movement direction, and the final component represents whether or not to jump.

To provide dense supervision to the RL optimization process, we use a reward function that gives a small reward on every time step that the agent decreases its minimum distance to the goal achieved thus far on the current obstacle course, as well as a large reward for completing the task by reaching the obstacle end goal. Our reward function for all obstacle courses is:

- Reward of $+0.05 \times (\text{previous minimum distance} - \text{new minimum distance})$ each time a new minimum distance is achieved.
- Reward of $+10$ (and the episode terminates) for touching the goal area with any part of the character.

We further facilitate learning by randomly sampling the agent position at the beginning of a training episode from safe regions within the obstacle. Safe regions are defined as areas on top of blocks that do not damage the player.

ALGORITHM 1: Policy-Selection Go-Explore

```

1 Procedure EXPLOREPHASE( $s_{start}$ ):
2   Input:  $s_{start}$  is the initial high-dimensional simulator state. Output:  $\mathcal{A}$  is the archive of all interestingly different bins and their best trajectories
   found.
3    $\mathcal{A} \leftarrow \{\text{bin}(s_{start}) \mapsto (s_{start}, R = 0, L = 0, \text{stats.init}())\}$       /*  $\triangleright$   $R$ : reward,  $L$ : trajectory length, bin: binning function */
4   while not SOLVED do
5      $c \leftarrow \text{SELECTBIN}(\mathcal{A})$       /*  $\triangleright$  Select bin using novelty-based sampling */
6      $s \leftarrow \mathcal{A}[c].\text{state}$       /*  $\triangleright$  Retrieve saved simulator state of bin  $c$  */
7     SETSIMULATORSTATE( $s$ )      /*  $\triangleright$  Restore exact state to avoid stochastic derailment */
8      $\text{policy} = \text{Random}$ 
9     if  $\text{stats.times\_chosen\_since\_update} == 0$  then
10      |  $\text{policy} = \text{PPO-Policy}$ 
11     end
12      $\text{stats.times\_chosen\_since\_update} \leftarrow \text{stats.times\_chosen\_since\_update} + 1$ 
13      $\text{stats.times\_chosen} \leftarrow \text{stats.times\_chosen} + 1$ 
14      $\mathcal{T}, \text{Solved} \leftarrow \text{EXPLORE}(s, \text{policy}, k = 200)$       /*  $\triangleright$  Exploration for  $k$  environment steps */
15     foreach  $(s', r_{cum}, l_{traj}) \in \mathcal{T}$  do
16       |  $c' \leftarrow \text{bin}(s')$ 
17       | if  $c' \notin \mathcal{A}$  then
18         | |  $\mathcal{A}[c'] \leftarrow (s', r_{cum}, l_{traj}, \text{stats.init}())$       /*  $\triangleright$  Add newly discovered bin to archive */
19       | end
20       | else
21         | |  $(s_{old}, r_{old}, l_{old}, \text{stats}) \leftarrow \mathcal{A}[c']$ 
22         | | if  $r_{cum} > r_{old} \vee (r_{cum} == r_{old} \wedge l_{traj} < l_{old})$  then
23           | | |  $\text{stats.times\_chosen\_since\_update} \leftarrow 0$ 
24           | | |  $\mathcal{A}[c'] \leftarrow (s', r_{cum}, l_{traj}, \text{stats})$       /*  $\triangleright$  Update if higher reward or shorter trajectory found */
25         | | end
26       | end
27       |  $\mathcal{A}[c']. \text{stats.times\_seen} \leftarrow \mathcal{A}[c']. \text{stats.times\_seen} + 1$ 
28     end
29   end

30 Function SELECTBIN( $\mathcal{A}$ ):
31   Input:  $\mathcal{A}$  is the current archive of explored bins. Output:  $c$  is a sampled bin chosen proportional to its novelty score.
32   return  $c \sim \frac{1}{\sqrt{(\mathcal{A}[c]. \text{stats.times\_seen} + 1)}}$ 

33 Function BINNINGFUNCTION( $s$ ):
34   Input:  $s$  is the high-dimensional environment state. Output: A low-dimensional discretized bin representation.
35    $x, y, z, \text{time}, \text{jumps\_used}, \text{interactive\_object\_pos} \leftarrow s$ 
36    $x\_bins \leftarrow 32$   $y\_bins \leftarrow 32$   $z\_bins \leftarrow 8$   $t\_bins \leftarrow 1$   $j\_bins \leftarrow 1$   $i\_bins \leftarrow 1$ 
37   if animated object then
38     |  $t\_bins \leftarrow 16$ 
39   end
40   if limited jumps then
41     |  $j\_bins \leftarrow \text{NumAllowedJumps}$       /*  $\triangleright$  If our queries constrain jumps, the limit is always less than 10. */
42   end
43   if interactive object then
44     |  $i\_bins \leftarrow 8$       /*  $\triangleright$  We bin only along forward (Y) axis of the obby. */
45   end
46   if animated object and limited jumps then
47     |  $x\_bins \leftarrow 16$   $y\_bins \leftarrow 16$ 
48   end
49   if animated object and interactive object then
50     |  $t\_bins \leftarrow 8$ 
51   end
52   if limited jumps and interactive object then
53     |  $z\_bins \leftarrow 4$ 
54   end
55   return  $(x\_bin, y\_bin, z\_bin, t\_bin, j\_bin, i\_bin)$ 

```

3.3 Alignment to the Roblox Engine (the Target Game Engine Platform)

The implementation of movement mechanics in our obstacle course games take inspiration from the character motion in Roblox. For grounded movement in Roblox, the character rapidly accelerates to full movement speed and immediately decelerates when stopping. When jumping, the character can influence its movement in the air, which is essential for solving some obstacle course challenges like Lava Ring Drop (in which the solution involves using air mobility to avoid touching lava blocks, see Sec. 7).

Inspired by this, we implement character directional motion (and jumping) as an external force applied to the character on the timestep the action is taken. With the exception of the jump force, this force can be applied whether the character is in the air or on the ground. We implement air resistance to control the character's maximum speed. We tuned each of these values such that movement speed, jump height, and jump distance were closely matched between our obstacle course games and Roblox.

Using our high performance simulator, we were able to automatically align physics behaviors by matching character motion trajectories in our simulator to those in Roblox. Specifically, we recorded trajectories of agent actions and positions using the agent deployed in Roblox as discussed in the previous section. We then loaded identical geometry into our simulator, and searched over physics parameters (using 8192 worlds in parallel) in our simulator to find the combination that best aligned the resulting positions when the action recorded from the Roblox trajectories were replayed. This process was repeated until a certain error threshold across the entire trajectory was reached. The physical parameters we optimized for were gravity, directional movement force, jump force, and air resistance.

4 Strategies for State Selection and Exploration

In this section, we present ablation studies that address three questions: a) What are alternative strategies for using a pretrained policy during the exploration phase? (Sec. 4.1) b) Can we leverage a pretrained *encoder* (as in [Gallouédec and Dellandréa 2023]) to restore more promising states during the return phase? (Sec. 4.2) c) How do pre-trained Reinforcement Learning (RL) based methods compare with Go-Explore? (Sec. 4.3)

In the next two sections, we compare the methods on a wall-clock time budget of 5 minutes. This allows comparison of the methods based on sample efficiency, measured as the number of "Go" steps (Alg. 1, line 5) where each go step performs 1.6M timesteps of simulation. We also compare solve rate (as described earlier) under this extended 5 minute time budget. As in the main paper, "Short", "Medium", and "Long" refer to the division of queries into tertiles based on the time taken to solve by Go-Explore.

4.1 Alternative methods for combining RL Policy and Go-Explore

We ablate the choice of exploration strategy using a pre-trained RL policy after returning to a previous state. Our main strategy is to use the policy for the duration of the exploration step when the restored state has not been explored from yet. We refer to this

strategy as Go-Explore-PolicySelection (GE-PS). In this section, we explore two additional strategies. The first strategy is to stochastically choose either the pre-trained RL policy or the default random exploration with 50% probability when a state is restored. This strategy is referred to as Go-Explore-RandomSelection (GE-RS). The second strategy is to *always* use the pre-trained RL policy during the exploration phase. We refer to this strategy as Go-Explore-RLPolicy (GE-RL).

In Fig. 2a, we find that all four strategies are solve nearly all queries in the Short and Medium tertiles. The differences emerge on Long queries: GE-PS reaches a 56% solve rate, substantially outperforming all other methods. If we compare GE-RS with GE-RL, both report identical solve rates and nearly identical mean steps to solve (Fig. 2b). The nearly identical results on both of these confirm that, though using the policy improves over the baseline GE, the same benefit could be achieved by using it fewer times, thus allowing more worlds to carry out random exploration. Our strategy of using the policy only when exploring from a state for the first time (GE-PS) is motivated by the observation that the policy with best performance in pretraining is *nearly deterministic* even on out-of-distribution environments like our novel obstacle course designs. Because our environments are deterministic up to thread-order in GPU physics simulation, this suggests that using the policy to explore from the same restored state multiple times is extremely unlikely to discover new bins—it is more effective to use the random policy in an attempt to find more novel states.

4.2 Effect of Pretrained Encoder on Cell-Free Latent Go-Explore

In Cell-free Latent Go-Explore (CFLGE), [Gallouédec and Dellandréa 2023] propose to train an encoder to map discovered states to a latent representation during exploration. States are sampled for exploration in low-density regions of this latent space. Within our 90 second time budget, training this encoder from scratch did not produce a benefit in average solution time over GE or our GE-PS (see Fig. 3, main paper). We therefore train an encoder offline using states and actions derived from the RL policy pretraining (ref. Sec. 3. Following [Gallouédec and Dellandréa 2023], we use the forward dynamics objective to train the encoder.

We ablate multiple variants of CFLGE using this pretraining process. First, we use this pretrained encoder with no online updates during Go-Explore search (CFLGE-Frozen). Second, we start from this pretrained encoder, but apply online updates using the forward dynamics objective during search (CFLGE-Finetune). These are both compared to training an encoder from scratch with online updates during search as in the original paper (CFLGE) [Gallouédec and Dellandréa 2023].

We find that in the Short tertile, CFLGE-Finetune improves the solve rate over CFLGE by 5% while CFLGE-Frozen degrades its performance by 20% (Fig. 3a). In the Medium tertile, all methods have comparable solve rates. Using the pretrained encoder without online updates (CFLGE-Frozen) is most advantageous in the Long tertile where it improves the solve rate by 7x over CFLGE (21% from 3%).

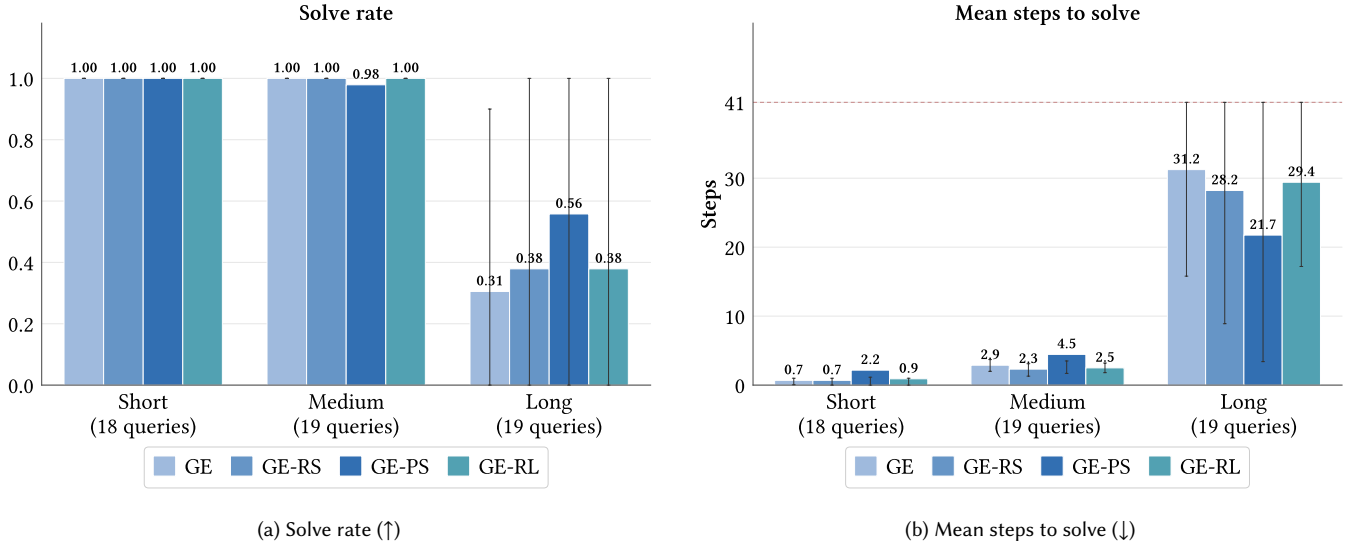


Fig. 2. Comparison of Go-Explore variants on solve rate and sample efficiency (measured as Go-Explore steps to solve): Go-Explore (GE), Go-Explore-RandomSelection (GE-RS), Go-Explore-PolicySelection (GE-PS), and Go-Explore-RLPolicy (GE-RL)

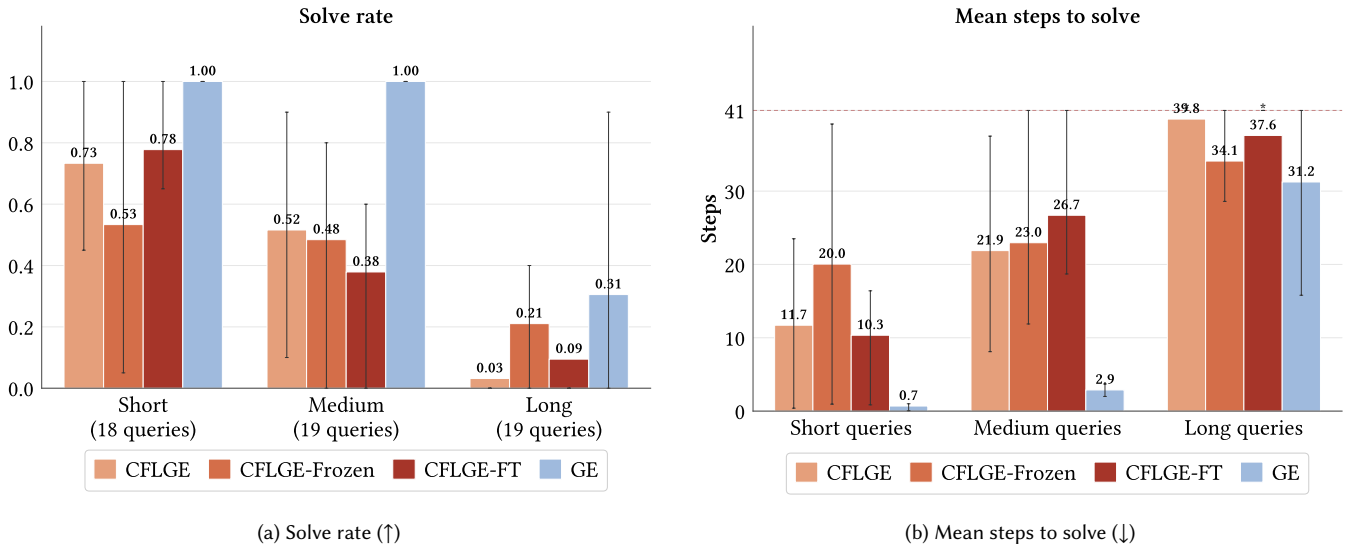


Fig. 3. Comparison of CFLGE variants against the Go-Explore baseline. CFLGE-Frozen substantially outperforms (7x improvement in Solve Rate) CFLGE and CFLGE-Finetune on Long queries, but all lag behind vanilla Go-Explore across all buckets.

Despite the favorable solve rate of CFLGE-Frozen in the Long tertile, we conclude that the pretrained encoding does not effectively restore favorable states for exploration in out-of-distribution obstacles. In retrospect, this is not surprising, as the generalization capability of the pretrained RL policy (training for which produced the state-action distribution as was used for training the encoder) was similarly limited (see Sec. 4.3 below). Fig. 3b shows the mean steps to solve over these methods, which inversely correlates with their solve rates.

4.3 Comparison of RL Policy with Go-Explore

We compare the two variants of purely RL-based methods in Fig. 4. In the first variant, we pretrain a policy using PPO [Schulman et al. 2017], as detailed in Sec. 3. We then run inference on this policy with no Go-Explore search on our query set as many times as possible within our time budget or until it solves the query 5 times. We then report solve rate (out of 5) and mean times to solve. We refer to this variant as “RL Policy”. In the second variant, we finetune this policy while running it on each of our queries by providing the reward

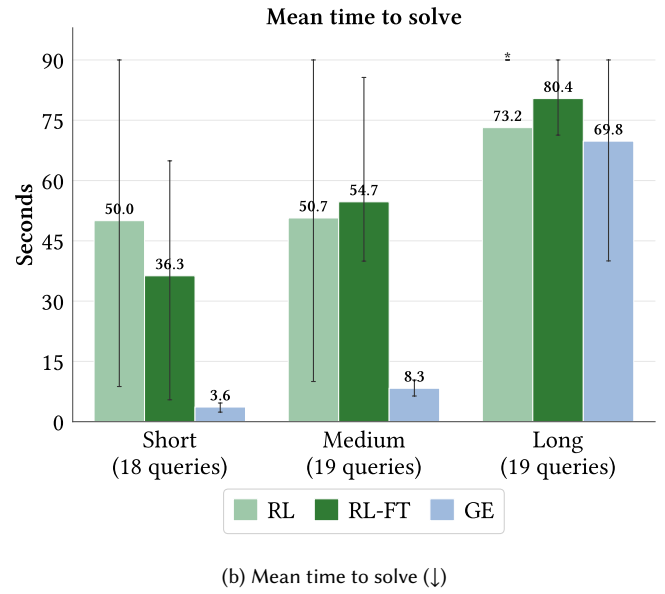
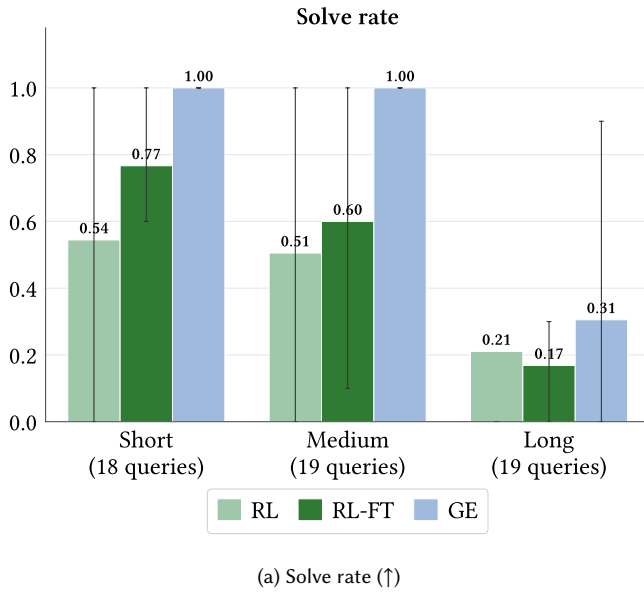


Fig. 4. Comparison of RL variants against the Go-Explore baseline. RL-Finetuning outperforms RL Policy on solve rates, but all lag behind vanilla Go-Explore across all buckets.

signal detailed in Sec. 3 for each step of the rollout. We refer to this variant as “RL Finetuning”. Any rollout reaching the goal during finetuning is counted as a success. We finetune across 5 seeds and report the average solve rate and mean time to solution.

RL Finetuning has a higher solve rate than RL Policy on Short tertile queries (0.77 vs 0.54, Fig. 4a). Both have similar performance in the Medium tertile, while in the Long tertile finetuning slightly degrades the solve rate (0.17 vs 0.21). We hypothesize that, on Short tertile queries, the RL Finetuning is more effective because the dense reward is more likely to lead the agent towards the goal and a solution is likely to be shorter. In the Long queries, the solution is more complex and the dense reward may lead the agent astray (e.g. by leading the agent directly towards the goal when the true solution is more roundabout). That is, the finetuning can improve a suboptimal solution on a short query, but often *unlearns* useful properties of that solution on a Long query, and cannot re-learn a better solution within the time budget. We observe an analogous trend for variants of CFLGE (see Fig. 3). Across all queries, Go-Explore outperforms both RL Policy and RL Finetuning.

5 Policy-Based and Cell-Free Latent Go Explore Implementation Details

We present details of our implementations of Policy-Based Go-Explore and Cell-Free Latent Go-Explore. Source code is given on the project website at www.dummyurl.com.

5.1 Policy-Based Go-Explore

Our implementation of Policy-Based Go-Explore (PBGE) is based on details given in the Methods section of [Ecoffet et al. 2021], and Sec. 14 of the supplement to that paper. We integrate PBGE into our existing G-oExplore implementation. Binning is identical to

our Go-Explore implementation, and is computed according to our bin resolution heuristics. At each “go” step, a bin is sampled using the selection weights given for PBGE (different from the default Go-Explore). We use the bin trajectory leading to the selected bin to present intermediate goals to the agent. Following the original paper, we use a window size of 10. To represent the goal bin, the observation space is augmented with the normalized bin coordinates of the goal bin along each of the 6 coordinate dimensions of our binning function. To align with the intuition that trajectory rewards should be lower than environment rewards, we provide a reward of 0.01 for each intermediate goal point and a reward of 0.03 for each end goal point (compare to the 0.05 dense reward for making progress towards the obstacle goal, described in Sec. 3.2).

We implement the policy loss function as described in Sec. 14 of the supplement to [Ecoffet et al. 2021]. We use 6.25% of the 8K parallel worlds as self-imitation learning (SIL) worlds—these are the only worlds which contribute to the SIL term of the loss function. We increase the entropy by dividing the logits output by the policy following the schedule of Eq. 8 of [Ecoffet et al. 2021], using the same constant values, and similarly terminating the episode after 1000 steps if the next goal bin (while returning) or a new bin (while exploring) has not been found.

5.2 Cell-Free Latent Go-Explore

Our implementation of Cell-free Latent Go-Explore (CFLGE) is based on the “Latent Go-Explore” section in the paper [Gallouédec and Dellandréa 2023]. However, we adapt it to suit our use case which is different from CFLGE [Gallouédec and Dellandréa 2023] in two ways: a) we can restore from state (without requiring a goal-conditioned policy), and b) we operate under a time constraint in order to provide rapid feedback. We also introduce a memory buffer for bookkeeping

and replace the goal-conditioned policy learned to return with direct states restored in the simulator. We discuss these modifications next.

Encoder training. Following Go-Explore [Ecoffet et al. 2021], "go" step samples a state and "explore" step explores from that state according to an exploration policy. The difference in CFLGE as compared to Go-Explore is that states are sampled during the "go" step inversely proportional to density in a learned latent space. Unlike CFLGE [Gallouédec and Dellandréa 2023], which trains the latent encoder after every few "go" steps, we continuously train the latent encoder at every simulation step. Every simulation step generates 8K transitions to train the encoder on. Using the forward dynamics objective, we train the encoder on this data for 4 epochs, containing 4 minibatches per epoch. The number of stored states is limited by available GPU memory. We describe our strategy for managing stored states below.

Memory Buffer Management. Each "go" step runs 8,192 worlds in parallel for 200 "explore" steps, so the simulator produces $8192 \times 200 \approx 1.6\text{M}$ states per go step. Each stored state is a simulator checkpoint (24 KB) together with its latent (256 floats = 1 KB), $\approx 25\text{KB}$ in total. On a single RTX4090 (24 GB VRAM), we dedicate ≈ 2.9 GB to the state buffer; at ≈ 25 KB per state that budget holds $\approx 120\text{k}$ states. Note that 120k states is only about 15 explore steps of generation: 15 steps $\times 8192$ worlds $\approx 120\text{k}$ states, versus the 200 explore steps in a single go step. The buffer therefore saturates within the 15 "explore" steps, and every subsequent batch of 8192 new states must displace an equal number of stored slots.

When managing saved states, we wish to evict older states in order to admit newly discovered states while also ensuring that states in sparse regions of the latent states remain restorable for future exploration. To achieve this, we cluster the latent codes of all existing states with k -means++ ($k = 2400$, one centroid per ~ 50 slots) and exempt the smallest 20%-ile of non-empty clusters (480 clusters) from eviction. We then manage the remaining states with a simple FIFO policy. k -means++ is run once, at bootstrap, costing ≈ 3.0 s. Centroids drift incrementally via running-mean updates as states enter and leave the buffer.

Density Estimation Density estimation is performed over the latents in the buffer during the "go" step. We perform kNN to estimate density as in [Gallouédec and Dellandréa 2023]. To optimize the kNN computation, we independently subsample the buffer into a reference subset and a candidate subset, each drawn uniformly at random from the occupied slots. The reference subset is capped at $N_{\text{ref}} = 25,000$ latents (which bounds the k -NN distance matrix), and the candidate subset at N_{cand} slots. Each candidate's density is estimated as the (negative) distance to its k -th nearest reference neighbor ($k = 1,000$). Candidates are then sorted by ascending density (most novel first), and a restart slot is drawn by sampling a rank from a geometric distribution ($p = 0.005$) truncated to the list length, following [Gallouédec and Dellandréa 2023].

6 Per-Query Quantitative Results

In this section, we include results for all individual queries in our query set. Table 1 gives the wall-clock time to first solution for all compared methods across all queries. Table 3 gives the solve rate out of 5 trials of all methods. Finally, Table 2 gives the number of "Go"

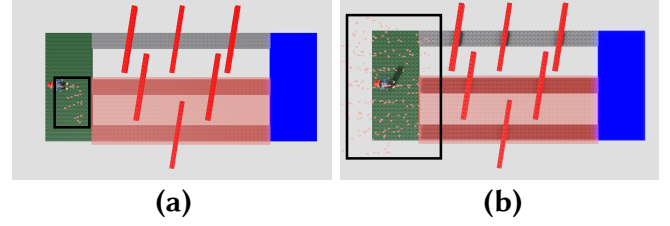


Fig. 5. The pink dots visualize saved states after two explore steps on the 123SpinnerPlank/10_9_42_1 query under (a) our Go-Explore Policy-Selection (GE-PS) and (b) Go-Explore with random exploration (GE). On this query, policy behavior is initially slow to explore, leading to a smaller explored area than a randomly exploring policy and a total avg. delay of 8s in the time to first solution.

steps in the Go-Explore algorithm taken to reach the first solution for all applicable methods.

For Hard queries (Table 3), our Go-Explore Policy-Selection method (GE-PS) accelerates the time to first solution and increases the solve rate across all queries. To illustrate why a single use of the policy can incur a slight performance penalty, Fig 5 shows an example query after two "Go" steps under our Go-Explore Policy-Selection and standard Go-Explore. The trained policy initially explores a smaller area than a random policy, leading to an increased average time to first solution.

In Table 1, 2, and 3 each query is named after the level it was authored on, followed by its index within that level: Designer1_Query3 is the third query posed on the DESIGNER1 level. Levels authored in a single interactive session by one of our three designers are anonymized as DESIGNER1–DESIGNER3; the remaining levels are named for the obstacle they are built around. Queries sharing a prefix are therefore posed on the same base level, and differ only in the obstacles added and in the waypoint and avoid-region constraints that constitute the query itself.

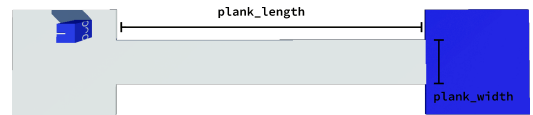
7 Procedural Obstacle Game Environment Generation

We train our policy on procedurally generated environments. Our procedurally generated obstacles were designed to model common design patterns and idioms seen in published obstacle course games on popular game platforms (e.g., traversing long walkways, jumping between blocks with air gaps in between, jumping over a wall).

Our procedural system organizes design patterns into a fixed set of parameters obstacle segment types. Below is a description of each parameterized obstacle type. For each, we include its parameters and a visualization of how those parameters correspond to the realized obstacle.

Plank

A plank is a straight platform from the spawn point to the goal.



Query	Mean time to first solution (seconds) – 90 s timeout								
	GE	GE-RS	GE-PS	GE-RL	CFLGE	CFLGE-Frozen	CFLGE-FT	RL	RL-FT
<i>Short (18 queries)</i>									
Designer1_Query2	1.24	1.70	1.70	1.70	16.96	15.20	17.08	7.00	4.50
Designer1_Query1	1.48	2.10	3.10	2.10	31.01	38.90	42.11	7.00	4.70
RunToSafety_Query3	2.22	3.20	3.10	3.20	90.00	90.00	90.00	77.40	86.30
Designer1_Query3	2.30	2.70	4.30	3.00	34.86	90.00	75.75	11.00	13.20
3SpinFwd_Query3	2.36	3.30	3.60	3.40	41.17	43.80	59.59	46.40	5.90
3SpinFwd_Query1	2.44	3.30	3.20	3.30	45.87	44.70	43.28	7.00	2.50
3SpinFwd_Query2	2.46	3.20	3.20	3.20	38.40	43.60	75.30	7.00	2.50
Designer1_Query5	3.13	4.20	3.70	4.50	41.98	90.00	42.19	8.00	12.60
Designer1_Query4	4.20	3.40	6.80	4.70	80.45	90.00	90.00	12.00	24.70
123SpinPlank_Query3	4.26	4.90	5.60	5.00	84.92	82.60	74.66	90.00	5.30
123SpinPlank_Query5	4.33	6.70	7.60	7.00	90.00	90.00	89.97	90.00	32.70
123SpinPlank_Query1	4.40	6.20	6.70	7.00	66.66	90.00	73.01	90.00	37.10
Designer3_Query1	4.63	5.80	5.30	6.30	90.00	90.00	90.00	75.40	49.90
Designer1_Query6	4.66	4.90	5.20	5.30	90.00	90.00	90.00	90.00	90.00
123SpinPlank_Query2	4.77	7.80	15.50	7.80	69.34	90.00	87.67	90.00	51.70
RidePlatDoor_Query2	5.31	12.00	77.10	25.10	90.00	90.00	88.65	12.00	69.30
SpikyLogRoll_Query1	5.72	5.20	5.50	5.90	90.00	90.00	90.00	90.00	83.70
Designer3_Query2	5.73	6.60	8.30	7.60	90.00	90.00	89.52	90.00	76.70
Short mean	3.65$\pm$1.45	4.84$\pm$2.47	9.42$\pm$17.17	5.89$\pm$5.15	65.65$\pm$26.09	74.93$\pm$24.87	72.71$\pm$22.46	50.01$\pm$39.24	36.29$\pm$32.72
<i>Medium (19 queries)</i>									
123SpinPlank_Query4	5.85	6.90	7.30	9.00	90.00	90.00	90.00	90.00	5.60
RunToSafety_Query1	5.87	6.30	6.20	6.10	90.00	90.00	90.00	61.00	54.10
Designer2_Query2	6.06	6.80	12.40	7.80	90.00	90.00	90.00	90.00	4.50
PlatSlideCheck_Query2	6.09	5.10	5.00	5.20	81.26	79.50	79.41	9.00	16.60
SpikyLogRoll_Query2	6.23	5.70	5.50	6.50	90.00	90.00	90.00	90.00	81.30
SlashingLava_Query1	6.55	8.40	8.30	8.50	90.00	90.00	90.00	10.00	40.80
RidePlatDoor_Query1	6.72	12.20	39.70	12.50	90.00	90.00	90.00	12.00	28.90
MultiPlatRide_Query3	6.90	8.00	7.90	8.10	90.00	90.00	90.00	10.00	57.00
Designer2_Query4	7.15	7.00	8.10	6.90	90.00	90.00	90.00	90.00	39.10
123SpinPlank_Query6	7.26	10.60	10.80	11.30	90.00	90.00	90.00	90.00	42.00
SineWave_Query1	7.48	5.20	5.20	4.90	90.00	90.00	90.00	7.00	47.70
Designer2_Query1	8.20	8.80	9.20	9.30	90.00	90.00	90.00	90.00	90.00
MultiPlatRide_Query1	8.54	10.80	10.50	10.40	90.00	90.00	90.00	10.00	90.00
MultiPlatRide_Query2	9.20	13.60	15.50	13.00	90.00	90.00	90.00	11.00	90.00
Designer3_Query6	11.54	14.90	13.10	19.40	90.00	90.00	90.00	9.00	51.90
RunToSafety_Query2	11.57	10.60	10.70	9.70	90.00	90.00	90.00	90.00	90.00
RidePlatDoor_Query3	11.60	16.20	75.60	15.00	90.00	90.00	90.00	14.00	68.50
RevSpinLily_Query1	12.05	15.20	20.70	18.70	90.00	90.00	90.00	90.00	90.00
TwoWaySlowSpin_Query1	13.17	14.40	14.40	15.20	90.00	90.00	90.00	90.00	51.30
Medium mean	8.32$\pm$2.44	9.83$\pm$3.68	15.06$\pm$16.64	10.39$\pm$4.27	89.54$\pm$2.01	89.45$\pm$2.41	89.44$\pm$2.43	50.68$\pm$39.99	54.70$\pm$28.72
<i>Long (19 queries)</i>									
Designer3_Query3	16.94	15.40	13.50	18.00	90.00	90.00	90.00	8.00	56.20
Designer3_Query5	18.14	17.80	12.40	20.10	90.00	90.00	90.00	9.00	55.30
SineWave_Query2	20.47	6.90	6.70	6.30	90.00	90.00	90.00	90.00	90.00
TwoWaySlowSpin_Query2	20.70	12.30	11.80	13.70	90.00	90.00	90.00	90.00	44.90
Hard_SpinnerTrussButton	26.54	65.00	36.50	63.80	90.00	90.00	90.00	90.00	90.00
SlashingLava_Query2	53.47	40.20	38.10	75.80	90.00	90.00	90.00	90.00	90.00
Hard_UpAndDownSmall	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00
Hard_BigOne	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00
Hard_UpAndDownFull	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00
Hard_UpAndDownMid	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00
Hard_RevSpinLily9Jumps	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00
RevSpinLily_Query3	90.00	90.00	59.20	90.00	90.00	90.00	90.00	90.00	90.00
RevSpinLily_Query2	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00
PlatSlideCheck_Query3	90.00	90.00	12.70	90.00	90.00	90.00	90.00	12.00	58.40
PlatSlideCheck_Query1	90.00	90.00	9.20	90.00	90.00	90.00	90.00	11.00	62.60
Designer2_Query3	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00
Hard_BlockButtonWallWrap	90.00	90.00	90.00	90.00	90.00	90.00	90.00	90.00	80.00
Designer3_Query4	90.00	84.90	65.30	80.30	90.00	90.00	90.00	90.00	90.00
RidePlatDoor_Query4	90.00	27.60	29.90	28.30	90.00	90.00	90.00	90.00	90.00
Long mean	69.80$\pm$31.40	66.32$\pm$33.41	53.44$\pm$35.56	68.23$\pm$32.21	90.00$\pm$0.00	90.00$\pm$0.00	90.00$\pm$0.00	73.16$\pm$33.52	80.39$\pm$15.77
Overall (56 queries)	27.68$\pm$35.45	27.39$\pm$34.18	26.27$\pm$31.41	28.57$\pm$34.34	82.02$\pm$18.46	84.97$\pm$15.55	84.25$\pm$14.90	58.09$\pm$38.54	57.50$\pm$31.83

Table 1. Mean time to first solution (seconds), per query. Complete per-query breakdown of the aggregate timings reported in Fig. 3 of the main paper and Fig. 4b.

Query	Mean “Go” steps to first solution – 41-step cap						
	GE	GE-RS	GE-PS	GE-RL	CFLGE	CFLGE-Frz	CFLGE-FT
<i>Short (18 queries)</i>							
Designer1_Query2	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Designer1_Query1	0.0	0.0	0.0	0.0	0.0	0.0	0.0
RunToSafety_Query3	0.0	0.0	0.0	0.0	19.0	41.0	19.0
Designer1_Query3	0.0	0.0	0.0	0.0	0.2	41.0	2.4
3SpinFwd_Query3	0.0	0.0	1.0	0.0	0.4	0.8	0.8
3SpinFwd_Query1	0.2	0.0	0.0	0.0	0.8	0.8	0.2
3SpinFwd_Query2	0.2	0.0	0.0	0.0	0.4	0.8	1.6
Designer1_Query5	1.0	1.0	1.0	1.0	0.0	41.0	0.0
Designer1_Query4	0.6	0.0	1.0	0.2	25.0	33.0	41.0
123SpinPlank_Query3	1.0	1.0	1.0	1.0	1.2	1.4	1.0
123SpinPlank_Query5	1.0	1.0	1.6	1.0	2.4	41.0	18.0
123SpinPlank_Query1	1.0	1.0	1.2	1.0	1.0	33.2	1.2
Designer3_Query1	1.0	1.0	1.0	1.0	2.6	26.8	41.0
Designer1_Query6	1.0	1.0	1.0	1.0	33.4	41.0	2.8
123SpinPlank_Query2	1.0	1.4	4.0	1.2	1.4	11.2	2.4
RidePlatDoor_Query2	2.0	3.0	23.2	7.0	41.0	3.6	11.6
SpikyLogRoll_Query1	1.2	1.0	1.0	1.0	41.0	33.6	41.0
Designer3_Query2	1.2	1.0	2.0	1.0	41.0	10.6	2.2
Short mean	0.7$\pm$0.6	0.7$\pm$0.8	2.2$\pm$5.3	0.9$\pm$1.6	11.7$\pm$16.6	20.0$\pm$17.9	10.3$\pm$15.3
<i>Medium (19 queries)</i>							
123SpinPlank_Query4	1.8	1.2	1.4	1.8	18.8	41.0	41.0
RunToSafety_Query1	2.0	1.0	1.0	1.0	4.8	41.0	26.2
Designer2_Query2	2.0	1.4	3.4	1.8	41.0	26.4	33.8
PlatSlideCheck_Query2	2.0	1.0	1.0	1.0	1.6	1.0	1.0
SpikyLogRoll_Query2	1.6	1.0	1.0	1.0	41.0	41.0	34.0
SlashingLava_Query1	2.0	2.0	2.0	2.0	3.4	2.0	2.0
RidePlatDoor_Query1	2.6	3.2	11.6	3.4	19.4	11.6	33.2
MultiPlatRide_Query3	2.0	2.0	2.0	2.0	2.8	18.0	41.0
Designer2_Query4	2.8	2.0	2.0	2.0	41.0	3.2	26.2
123SpinPlank_Query6	2.4	2.2	2.8	2.6	33.8	41.0	33.4
SineWave_Query1	3.0	1.0	1.0	1.0	11.4	3.6	3.2
Designer2_Query1	2.4	2.0	2.0	2.0	41.0	41.0	41.0
MultiPlatRide_Query1	3.2	3.0	3.0	3.0	3.4	12.6	4.4
MultiPlatRide_Query2	4.0	4.0	4.0	4.0	12.4	12.2	12.0
Designer3_Query6	4.4	3.8	3.6	5.0	41.0	34.2	41.0
RunToSafety_Query2	4.6	2.4	2.6	2.6	26.2	41.0	41.0
RidePlatDoor_Query3	3.4	4.0	31.8	3.6	26.6	12.2	25.4
RevSpinLily_Query1	4.2	3.8	5.4	4.6	34.0	41.0	41.0
TwoWaySlowSpin_Query1	4.4	3.0	3.0	3.0	12.2	12.2	26.6
Medium mean	2.9$\pm$1.0	2.3$\pm$1.1	4.5$\pm$7.0	2.5$\pm$1.2	21.9$\pm$15.3	23.0$\pm$16.2	26.7$\pm$14.8
<i>Long (19 queries)</i>							
Designer3_Query3	6.4	4.0	3.6	4.8	41.0	33.6	41.0
Designer3_Query5	7.0	4.6	3.2	5.4	41.0	26.8	41.0
SineWave_Query2	8.8	1.4	1.4	1.4	41.0	37.2	41.0
TwoWaySlowSpin_Query2	6.8	3.0	3.0	3.0	41.0	19.8	12.4
Hard_SpinnerTrussButton	10.0	27.4	9.8	27.2	41.0	41.0	41.0
SlashingLava_Query2	21.6	10.4	10.0	27.0	41.0	30.2	41.0
Hard_UpAndDownSmall	41.0	41.0	41.0	41.0	41.0	41.0	41.0
Hard_BigOne	41.0	41.0	41.0	41.0	41.0	41.0	41.0
Hard_UpAndDownFull	41.0	41.0	41.0	41.0	41.0	41.0	41.0
Hard_UpAndDownMid	41.0	41.0	41.0	41.0	41.0	41.0	41.0
Hard_RevSpinLily9Jumps	41.0	41.0	41.0	41.0	41.0	41.0	41.0
RevSpinLily_Query3	41.0	41.0	22.6	41.0	41.0	41.0	41.0
RevSpinLily_Query2	41.0	41.0	41.0	41.0	41.0	41.0	41.0
PlatSlideCheck_Query3	41.0	41.0	3.0	41.0	41.0	3.6	12.0
PlatSlideCheck_Query1	41.0	41.0	2.0	41.0	18.6	18.6	33.8
Designer2_Query3	41.0	41.0	41.0	41.0	41.0	41.0	41.0
Hard_BlockButtonWallWrap	41.0	41.0	41.0	41.0	41.0	41.0	41.0
Designer3_Query4	41.0	27.0	18.4	31.6	41.0	41.0	41.0
RidePlatDoor_Query4	41.0	7.4	8.2	7.4	41.0	27.0	41.0
Long mean	31.2$\pm$15.1	28.2$\pm$16.7	21.7$\pm$17.7	29.4$\pm$16.1	39.8$\pm$5.1	34.1$\pm$10.6	37.6$\pm$9.1
Overall (56 queries)	11.8$\pm$16.5	10.6$\pm$16.0	9.6$\pm$14.3	11.1$\pm$16.2	24.7$\pm$17.5	25.8$\pm$16.1	25.1$\pm$17.2

Table 2. Mean “Go” steps to first solution, per query. Complete per-query breakdown of the sample-efficiency comparisons in Figs. 2b and 3b; each Go step performs 1.6M timesteps of simulation.

Query	Solve rate (fraction of 5 seeds solved)									
	GE	GE-RS	GE-PS	GE-RL	PBGE	CFLGE	CFLGE-Frozen	CFLGE-FT	RL	RL-FT
<i>Short</i> (18 queries)										
Designer1_Query2	1.00	1.00	1.00	1.00	0.20	1.00	1.00	1.00	1.00	1.00
Designer1_Query1	1.00	1.00	1.00	1.00	0.20	1.00	1.00	1.00	1.00	1.00
RunToSafety_Query3	1.00	1.00	1.00	1.00	0.00	0.60	0.00	0.60	0.20	0.20
Designer1_Query3	1.00	1.00	1.00	1.00	0.20	1.00	0.00	1.00	1.00	1.00
3SpinFwd_Query3	1.00	1.00	1.00	1.00	0.20	1.00	1.00	1.00	1.00	1.00
3SpinFwd_Query1	1.00	1.00	1.00	1.00	0.00	1.00	1.00	1.00	1.00	1.00
3SpinFwd_Query2	1.00	1.00	1.00	1.00	0.20	1.00	1.00	1.00	1.00	1.00
Designer1_Query5	1.00	1.00	1.00	1.00	0.00	1.00	0.00	1.00	1.00	1.00
Designer1_Query4	1.00	1.00	1.00	1.00	0.00	0.40	0.20	0.00	1.00	1.00
123SpinPlank_Query3	1.00	1.00	1.00	1.00	0.00	1.00	1.00	1.00	0.00	1.00
123SpinPlank_Query5	1.00	1.00	1.00	1.00	0.00	1.00	0.00	0.60	0.00	1.00
123SpinPlank_Query1	1.00	1.00	1.00	1.00	0.20	1.00	0.20	1.00	0.00	0.80
Designer3_Query1	1.00	1.00	1.00	1.00	0.00	1.00	0.40	0.00	0.60	0.80
Designer1_Query6	1.00	1.00	1.00	1.00	0.00	0.20	0.00	1.00	0.00	0.00
123SpinPlank_Query2	1.00	1.00	1.00	1.00	0.00	1.00	0.80	1.00	0.00	0.60
RidePlatDoor_Query2	1.00	1.00	1.00	1.00	0.00	0.00	1.00	0.80	1.00	0.40
SpikyLogRoll_Query1	1.00	1.00	1.00	1.00	0.00	0.00	0.20	0.00	0.00	0.40
Designer3_Query2	1.00	1.00	1.00	1.00	0.00	0.00	0.80	1.00	0.00	0.60
Short mean	1.00$\pm$0.00	1.00$\pm$0.00	1.00$\pm$0.00	1.00$\pm$0.00	0.07$\pm$0.10	0.73$\pm$0.41	0.53$\pm$0.45	0.78$\pm$0.38	0.54$\pm$0.49	0.77$\pm$0.32
<i>Medium</i> (19 queries)										
123SpinPlank_Query4	1.00	1.00	1.00	1.00	0.20	0.60	0.00	0.00	0.00	1.00
RunToSafety_Query1	1.00	1.00	1.00	1.00	0.00	1.00	0.00	0.40	0.60	0.80
Designer2_Query2	1.00	1.00	1.00	1.00	0.00	0.00	0.40	0.20	0.00	1.00
PlatSlideCheck_Query2	1.00	1.00	1.00	1.00	0.00	1.00	1.00	1.00	1.00	1.00
SpikyLogRoll_Query2	1.00	1.00	1.00	1.00	0.00	0.00	0.00	0.20	0.00	0.20
SlashingLava_Query1	1.00	1.00	1.00	1.00	0.00	1.00	1.00	1.00	1.00	0.80
RidePlatDoor_Query1	1.00	1.00	1.00	1.00	0.00	0.60	0.80	0.20	1.00	1.00
MultiPlatRide_Query3	1.00	1.00	1.00	1.00	0.00	1.00	0.60	0.00	1.00	0.80
Designer2_Query4	1.00	1.00	1.00	1.00	0.20	0.00	1.00	0.40	0.00	1.00
123SpinPlank_Query6	1.00	1.00	1.00	1.00	0.00	0.20	0.00	0.20	0.00	1.00
SineWave_Query1	1.00	1.00	1.00	1.00	0.00	0.80	1.00	1.00	1.00	0.80
Designer2_Query1	1.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00
MultiPlatRide_Query1	1.00	1.00	1.00	1.00	0.00	1.00	0.80	1.00	1.00	0.00
MultiPlatRide_Query2	1.00	1.00	1.00	1.00	0.00	0.80	0.80	0.80	1.00	0.00
Designer3_Query6	1.00	1.00	1.00	1.00	0.00	0.00	0.20	0.00	1.00	1.00
RunToSafety_Query2	1.00	1.00	1.00	1.00	0.00	0.40	0.00	0.00	0.00	0.00
RidePlatDoor_Query3	1.00	1.00	0.60	1.00	0.00	0.40	0.80	0.40	1.00	0.40
RevSpinLily_Query1	1.00	1.00	1.00	1.00	0.00	0.20	0.00	0.00	0.00	0.00
TwoWaySlowSpin_Query1	1.00	1.00	1.00	1.00	0.00	0.80	0.80	0.40	0.00	0.60
Medium mean	1.00$\pm$0.00	1.00$\pm$0.00	0.98$\pm$0.09	1.00$\pm$0.00	0.02$\pm$0.06	0.52$\pm$0.41	0.48$\pm$0.43	0.38$\pm$0.39	0.51$\pm$0.50	0.60$\pm$0.43
<i>Long</i> (19 queries)										
Designer3_Query3	1.00	1.00	1.00	1.00	0.00	0.00	0.20	0.00	1.00	1.00
Designer3_Query5	1.00	1.00	1.00	1.00	0.00	0.00	0.40	0.00	1.00	0.60
SineWave_Query2	1.00	1.00	1.00	1.00	0.00	0.00	0.20	0.00	0.00	0.00
TwoWaySlowSpin_Query2	1.00	1.00	1.00	1.00	0.00	0.00	0.60	0.80	0.00	0.60
Hard_SpinnerTrussButton	1.00	0.40	1.00	0.60	0.00	0.00	0.00	0.00	0.00	0.00
SlashingLava_Query2	0.80	1.00	1.00	1.00	0.00	0.00	0.60	0.00	0.00	0.00
Hard_UpAndDownSmall	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Hard_BigOne	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Hard_UpAndDownFull	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Hard_UpAndDownMid	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Hard_RevSpinLily9Jumps	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
RevSpinLily_Query3	0.00	0.00	0.60	0.00	0.00	0.00	0.00	0.00	0.00	0.00
RevSpinLily_Query2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
PlatSlideCheck_Query3	0.00	0.00	1.00	0.00	0.00	0.00	1.00	0.80	1.00	0.40
PlatSlideCheck_Query1	0.00	0.00	1.00	0.00	0.00	0.60	0.60	0.20	1.00	0.40
Designer2_Query3	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Hard_BlockButtonWallWrap	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.20
Designer3_Query4	0.00	0.80	1.00	0.60	0.00	0.00	0.00	0.00	0.00	0.00
RidePlatDoor_Query4	0.00	1.00	1.00	1.00	0.00	0.00	0.40	0.00	0.00	0.00
Long mean	0.31$\pm$0.46	0.38$\pm$0.48	0.56$\pm$0.50	0.38$\pm$0.47	0.00$\pm$0.00	0.03$\pm$0.14	0.21$\pm$0.30	0.09$\pm$0.25	0.21$\pm$0.42	0.17$\pm$0.29
Overall (56 queries)	0.76$\pm$0.42	0.79$\pm$0.40	0.84$\pm$0.36	0.79$\pm$0.40	0.03$\pm$0.07	0.42$\pm$0.45	0.41$\pm$0.42	0.41$\pm$0.44	0.42$\pm$0.49	0.51$\pm$0.43

Table 3. Solve rate per query, measured under the 300 s (5-minute) timeout. Every value in this table is a 300 s solve rate, matching the extended budget used in Sec. 4.1–4.3 and plotted in Figs. 2a, 3a and 4a.

plank_width: float

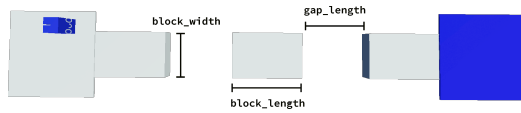
Describes the width of the plank.

plank_length: float

The length of the plank which a player must cross.

Path Jumps

A series of identical blocks uniformly spaced apart in a straight line to the goal.



num_blocks: int

The total number of blocks.

block_width: float

The width of each block.

block_length: float

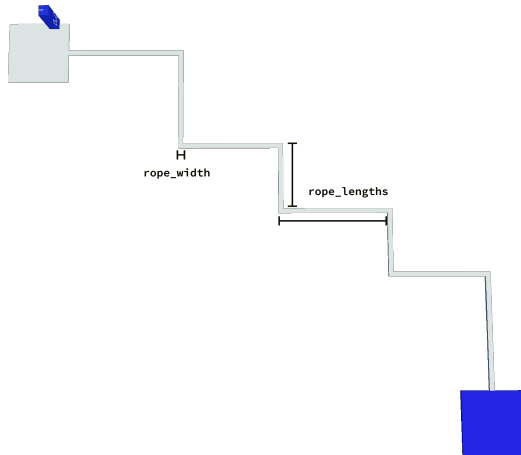
The length of each block.

gap_length: float

The size of the gap between each block.

Tightrope

A sequence of zigzagging platforms leading to the goal.



num_ropes: int

The number of rope segments.

rope_lengths: list[float]

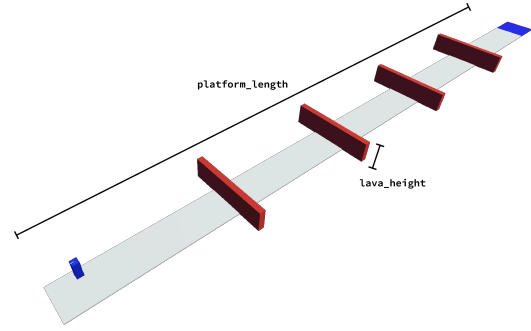
The length of each rope segment.

rope_width: float

The width of the rope segments.

Lava Hurdles

A platform with evenly spaced lava blocks of some height. The player must jump over the blocks to get to the goal.



num_hurdles: int

The number of lava hurdles.

lava_height: float

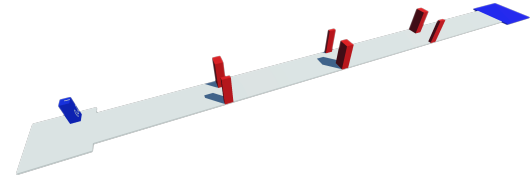
The height of each lava hurdle.

platform_length: float

The total length of the platform.

Lava Doors

A platform with two lava columns on the edges. The player move through the opening.



platform_length: float

The total length of the platform.

platform_width: float

The total width of the platform.

lava_height: float

The height of each lava column.

door_widths: list[float]

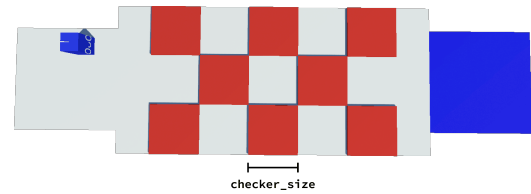
The widths of the opening between each door.

door_widths: list[float]

The position of the center of each opening (as a fraction of platform_width)

Lava Checkerboard

A checkerboard three squares wide and variable number of rows long. The checkerboards alternate between regular squares and lava squares.



num_rows: int

The number of checkerboard rows.

checker_size: float

The size of each checkerboard square.

Wall Wraps

A series of walls which require the player to maneuver around the walls in the air.



num_walls: int

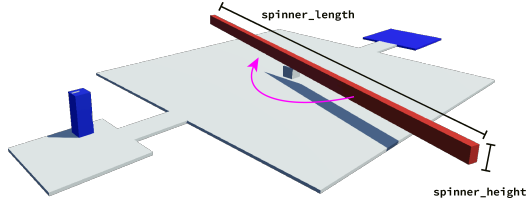
The number of walls to jump around.

wall_thickness: float

The thickness/depth of each wall.

Lava Spinner

A square platform with a one or two rotating lava spinner at the center.



num_spinners: (1,2)

The number of lava spinners.

platform_size: float

The dimension of the square platform.

Each spinner has its own:

spinner_height: float

The heights of the lava spinner.

spinner_length: float

The length of the lava spinner (diameter).

spinner_thickness: float

The width of the lava spinner.

angular_velocity: float

The speed at which the lava spinner rotates.

spinner_position: float

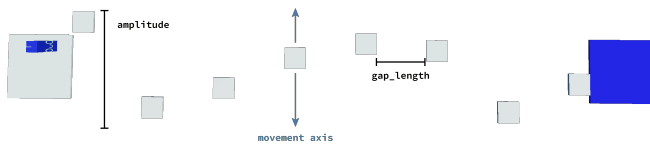
The position of the center of the spinner with respect to the platform.

starting_orientation: float

The orientation of the lava spinner at the start of an episode.

SideToSideBlocks

A series of horizontally moving blocks.



num_cases: int

Wedges

A straight platform with wedges across the platform.



num_wedges: int

The total number of wedges.

wedge_length: float

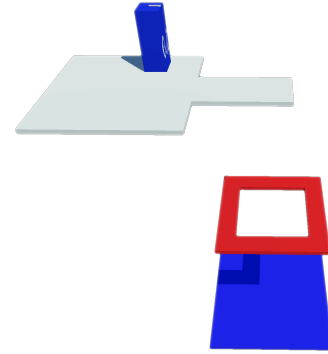
The length of each wedge.

wedge_height: float

The height of each wedge at its peak.

Lava Ring Drop

A ring of lava positioned above the goal.



goal_size: float

Side length of the goal.

goal_position: (float, float, float)

Position of the goal.

ring_border_thickness: float

Width of the lava ring border.

ring_hole_size: float

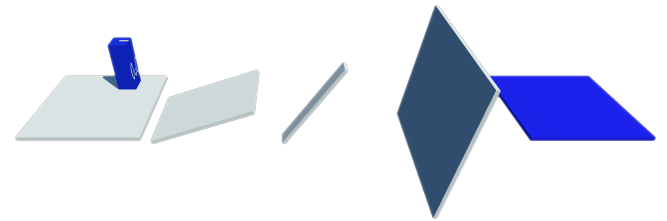
Side length of the lava ring hole.

ring_position: (float, float, float)

Position of the lava ring

Rotating Platforms

A series of three rotating platforms that the player must cross to reach the goal.

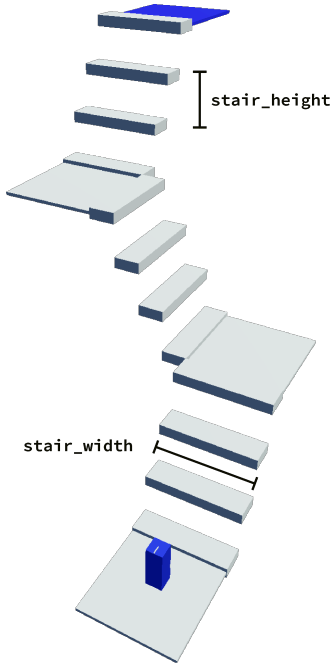


Each platform has its own:

size: (float, float)
 Length and width of the platform
 angular_velocity: float
 Angular velocity of the platform
 starting_orientation: float
 Starting orientation of the platform at the start of the episode

Staircase

A series of steps and platforms.



num_cases: int
 The number of chained staircases.
 num_stairs: int
 The number of stairs per staircase.
 stair_height: float
 The size of the vertical gap between each stair.
 stair_width: float
 The width of each stair.

8 Additional Design Case Studies

In this section we document additional illustrative design case studies and external designer studies.

8.0.1 Design Study 2: Exploring multiple solutions. In this study the designer wished to confirm that three diverse solution paths through an obstacle are possible to traverse (Fig. 6). These paths require different skills to traverse (climbing, navigating moving blocks, and dodging damaging obstacles).

The designer first queries for any solution and is provided one that travels along the path with the lava spinner obstacles (Fig. 6, (a), video 3:23). The designer now wishes to confirm that solutions exist along the other two paths. To do so, the designer uses an avoid

region to prohibit solutions along the already explored path. Now the system returns a solution along the path with moving blocks (Fig. 6, (b), video 3:43). The designer then verifies the middle path is also traversable by placing a waypoint at the highest point of the obstacle (Fig. 6, (c), video 3:55).

The designer also intended to allow the player to cross between these distinct paths by starting along one and then switching to another. To verify that this is possible, the designer places a waypoint halfway down the first path and an avoid region along the remaining ladder (Fig. 6, (d), video 4:19). The returned queries using these primitives show that it is possible to ascend halfway through the middle path and then jump down to the translating blocks to complete the level. Finally, the designer adds another waypoint to test if a solution can start on the middle path and switch to the lava spinner path before finishing the level, and our system returns that this solution exists (Fig. 6, (e), video 4:40).

8.0.2 Design Study 3: Adjusting difficulty. In this design (Fig. 7), the designer uses trajectory visualization to diagnose failure modes of their design and adjust the difficulty of consecutive level segments.

The designer first created a new section of the level with a lava spinner obstacle and test to see if it is solvable. When the system does not find a solution, the designer concludes from the trajectory visualizations that, in most failures, the player character is being hit by the lava spinner (Fig. 7, (a), video 5:41). They edit the obstacle to decrease the spinner speed and query to confirm that their edited obstacle is solvable.

The visualization of the solution shows that the character must make fast, precise jumps to get around the lava spinner (Fig. 7, (b), video 6:03). Based on this feedback, the designer concludes that this obstacle is difficult, so they design their next section with the explicit goal of making an easier obstacle. When the design is complete, they submit a query to check if the solution has properties that suggest it is easier than the previous section.

The designer observes that the returned solution shows the player character waiting on a block before jumping (Fig. 7, (c)). This is easier because the player does not have to make jumps in such quick succession, so the designer has met their goal for this section.

This study demonstrates that designers can go beyond the explicit feedback provided by the system by analyzing semantically meaningful properties of returned trajectories in order to draw conclusions about higher level properties (in this case, difficulty) of in-progress designs. Our user playtests for this obstacle confirm that users found the second section easier than the first (Sec. ??).

8.1 Additional Case Study 1: Evaluating multiple solution paths

The goal in this design is to create an obstacle with three paths featuring increasing difficulty of the same type of spinning lava block challenge. After creating the initial design, the designer wishes to know if solutions exist down each of the three paths and what those solutions look like.

The designer queries to see if the level is solvable along each of the intended paths. Our system returns multiple solutions that together demonstrate that a solution along each path exists (Fig. 8, (a)), answering the first part of the designer's question. The trajectory

visualizations further show that each path makes a lot of jumps. The designer now wishes to know which of these jumps are necessary for a solution.

To start answering this question, the designer queries for solutions with no jumping. The query shows that at least one path can be traversed without jumping at all. More queries confirm that this is true for all three paths (most difficult path visualized in Fig. 8, (b)). This demonstrates that, surprisingly, the answer to the designer's question is: no jumps are necessary for a solution. This is a counter-intuitive result that leads to interesting gameplay because it allows for less precise motions at the cost of much more precise timing. So, instead of attempting to edit the level to force jumping, the designer decides to leave the level as is.

The tool allowed the designer to verify properties of their initial design and revealed unexpected properties that changed the course of the design process. Note that it was only after seeing the first round of solutions that the designer formulated their refined queries—they did not know at the start that a solution without jumping was possible or desirable.

8.2 Additional Case Study 2: Evaluating a difficult obstacle

The designer sets out to create an extremely difficult obstacle that will likely require practice over multiple attempts to solve consistently. Precise motion and timing will be required to stay on the spinning spikes in the first half and then transition to avoiding them in the second half.

The designer does not know at start if a solution exists or how to analyze it. They first query to see a solution. The system returns that a solution *does* exist. The designer notices that the solution trajectory briefly waits on one of the initial spikes before jumping to the next, and hypothesizes that increasing the speed might make this impossible, and therefore make the obby impossible to solve. Another query tests this hypothesis.

The result is that the obstacle is still solvable—that is, while it may be impossible to wait on a spike, this fact does not make the obby impossible to solve (Fig. 9, left). Despite this, a view from the player camera causes the designer to conclude that the spikes are spinning too fast for a player to reason about. So, the designer edits the speed back down, this time to even slower than the original speed.

The designer now wishes to know if solutions that feature waiting on an initial spike exist. Another query shows that the agent can spend significant time (at least half a second) waiting on an intermediate spike to jump to a final one (Fig. 9, right). This is challenging and in line with the designer's goals, so it is the final design.

In this exercise, our system showed a solution even though the designer did not have an intended solution in mind or know how to reason about what properties of a solution helped or hindered their goals. After seeing a solution, they were able to do so even for this extremely difficult obby that they could not reliably playtest by hand.

8.3 Additional Case Study 3: Answering complex gameplay questions with queries

In this design, the designer intends the player to jump between a series of faster and faster oscillating blocks. The designer intends that the player should have time on each block to line up their jump if necessary. But, the obstacle should also be solvable in a faster, more fluid way, without as much waiting on slower blocks for the faster blocks to align—that is, the blocks should line up if the player starts crossing at the right time.

The designer queries to see if the blocks are correctly spaced and not moving too fast to jump to. The solution visualization says that both these properties are true: the player is able to jump between all blocks, and can wait on the third block before jumping to the fourth (Fig. 10, left).

The designer now tests to see if the more fluid solution is supported. Specifically, the designer wants to know: are the blocks moving in such a way that the player can traverse the obstacle staying on only one side of the center line? To answer this question, the designer sets up a query that forces solutions to avoid roughly half the span of each moving block. This query shows a solution that crosses nearly completely on one side of the centerline, and as a result features much less waiting on intermediate blocks (compare the length of trajectory spent standing on the third block in Fig. 10 left vs. right). This design meets the designer's goals.

Here, our system assisted the designer by allowing them to analyze a complex gameplay property. The query used to test this gameplay property is one which, without policy selection (Sec. 2), often does not find a solution, which might give misleading feedback to the designer.

8.4 External Designer 2

Designer 2 had not used Roblox before taking part in our study. To begin, they made a level with multiple blocks spaced much further apart than the character could jump. Feedback from our system showed that the first large jump was impossible, which prompted the designer to rethink their design for all subsequent jumps (Fig. 11, top left and center).

After confirming that this design was solvable (Fig. 11, top center), the designer wished to make the ending more challenging. To do so, they added two lava spinner obstacles above the final goal point (Fig. 11, top right). Our system confirmed that this design was solvable, but the designer concluded that, given the speed of the spinners, the solution shown involved "more luck than skill". They removed one of the spinners, reduced the speed of the remaining one, and queried for another solution.

Our system showed that the new design was solvable, but the solution skipped a block (Fig. 11, bottom left). The designer found this solution acceptable, but wished to further verify that the solution from the central block was also possible. To facilitate this, the designer added a moving ladder obstacle, but feedback from our system showed that the initial speed was too fast for the character to hold on to (Fig. 11, bottom center). In the final design, the designer decreased the speed of the moving ladder. They used our system to verify that the entire design was solvable from start to finish along their desired solution path (Fig. 11, bottom right).

During the design study, our system allowed an experienced designer to quickly acclimate to Roblox character control and physics while verifying solvability throughout the design. The visualized trajectories allowed the designer to determine which edits would move them closer to their design goals (ending spinners, midway moving ladder) even though the tool does not explicitly provide such feedback. Early in the design exercise, the designer often playtested level sections themselves. However, the more they used our interactive system, the more they began to trust it to answer their solvability queries. The designer made extensive use of the trajectory endpoint visualization (pink dots in the supplemental video) to place blocks in sequence.

8.5 External Designer 3

The designer articulated a starting goal of “something like running down a hallway with things flying at you”. As a first pass, they started with a thin walkway and added spinning lava obstacles to it (Fig. 12, (a)).

The designer’s initial goal was to find the point at which the obstacle could not be solved and “work backwards” from there. They ran a query to determine if their initial configuration was solvable, and found that a solution existed but was too difficult because of the high number of jumps over lava obstacles. Next, the designer wondered if it would be easier to ride a larger, slower spinner to a faraway goal instead of avoid smaller, faster spinners.

After modifying the design, the designer made two queries. First, they verified that it was possible to climb the starting walkway to get to a “pickup point” for the spinner (Fig. 12, (b)). Second, they verified that it was possible to ride the spinner from the pickup point to the goal (Fig. 12 (c)). Finally, the designer suspected that it should be possible to jump to the larger spinner from the starting block without going along the walkway. Using a query, they confirmed that this exploit exists (Fig. 12, (d)).

As with Designer 1, Designer 3 stated that the most important feedback was looking at the trajectories (rather than learning whether a design was solvable). This was particularly important even if the designer ended the search before a successful trajectory was found—often, the initial visualization told them what they needed to know.

8.6 External Designer 4

The designer wanted to make a level that increased in difficulty and showed off new mechanics as a good progression for a new player. After exploring our procedurally generated obstacles (Sec. 7), they started by created a sequence of moving blocks. The designer first wanted to know how close the third block in the sequence would need to be to ensure solvability. They queried our system and used the returned trajectory visualization to calibrate the maximum possible jump distance.

Reasoning that the maximum possible jump distance would be difficult for new players, the designer then created a conveyor belt obstacle leading into a descending door activated by standing on the yellow square (Fig. 13, (a), center). When querying for a solution to this configuration, our system returned an unexpected solution that jumped entirely over the second moving block to reach the purple conveyor (Fig. 13). This inspired the designer to make the skipped

block a lava obstacle in order to slightly increase the challenge and introduce lava as a concept to new players.

As with previous designers, the designer used our tool to acclimate to the movement controls of a new engine and to take inspiration about possible solutions on their designed level. The designer noted that the tool was useful for discovering unconventional ways a player might solve their level. They also said that because it is often difficult for designers to gauge difficulty for a new player, a built-in difficulty metric would be useful for calibration. We are excited to explore this for 3D obstacle course games in future work.

References

- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. 2021. First return, then explore. *Nature* 590, 7847 (Feb. 2021), 580–586. <https://doi.org/10.1038/s41586-020-03157-9>
- Quentin Gallouédec and Emmanuel Dellandréa. 2023. Cell-free latent go-explore. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) (*ICML ’23*). JMLR.org, Article 426, 16 pages.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *CoRR* abs/1707.06347 (2017). [arXiv:1707.06347](https://arxiv.org/abs/1707.06347)

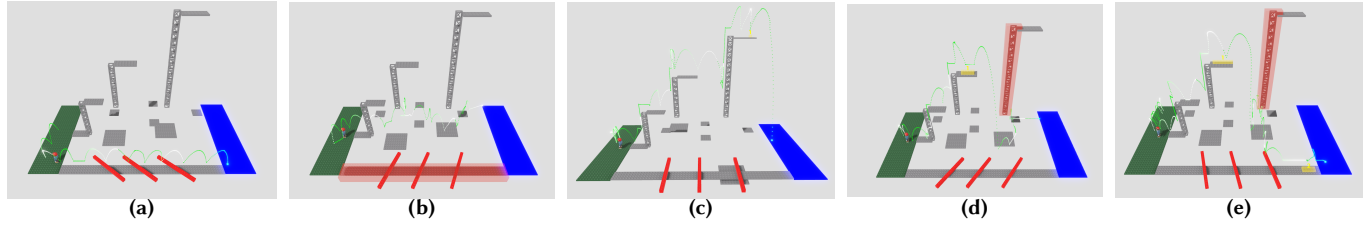


Fig. 6. Design study 2: Using a sequence of queries with waypoint and avoid region constraints to verify three distinct solution paths, each emphasizing different skills. After finding a solution for one path (a), the designer used an avoid region to find a new solution that avoided the original path (b). They repeated the process, but with a waypoint constraint at the top of the tower, to force the system to produce a solution for the third path (c). The designer also verified the presence of less obvious solutions that switch between the visually obvious solution paths ((d), (e)).

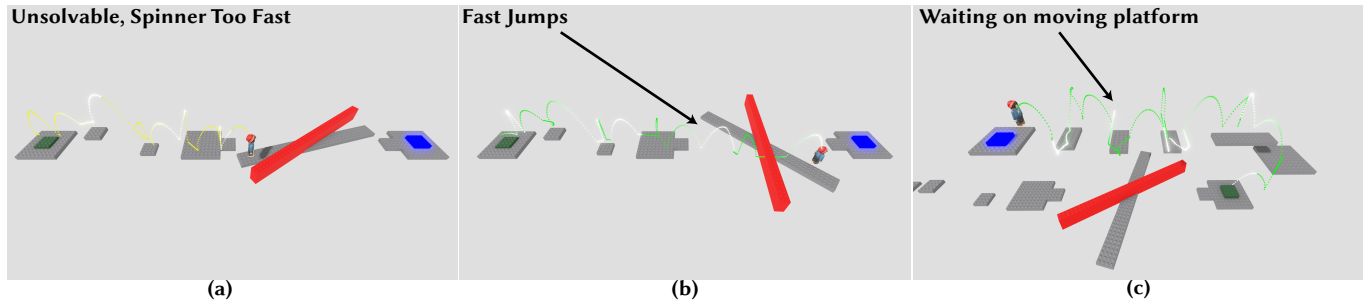


Fig. 7. Design study 3: The designer uses analysis of trajectory visualizations to control the difficulty of their level. (a) The designer realizes that their design has no solution because the lava spinner is spinning too fast for the player to cross. (b) A slower spinner admits a solution. The fast, precise jumps in the solution trajectory indicate that the solution is challenging. (c) Knowing that the spinner is a challenging segment, in the next section of this level, the designer constructs an easier challenge that gives the player more time to prepare before making each jump.

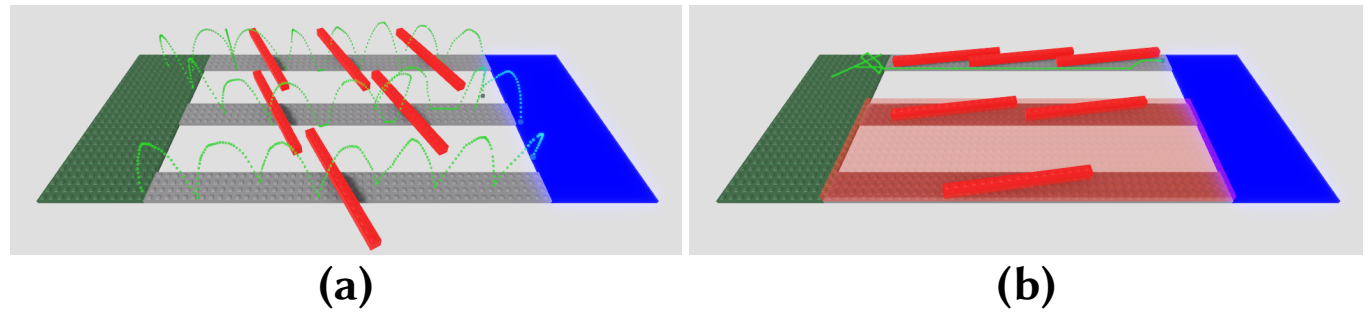


Fig. 8. Our query primitives enable verification of multiple solution paths. (a) Solutions along all three paths of the obstacle are verified simultaneously. (b) Using an avoid region with a jump count allows the designer to verify that the most difficult path is traversable without jumping.

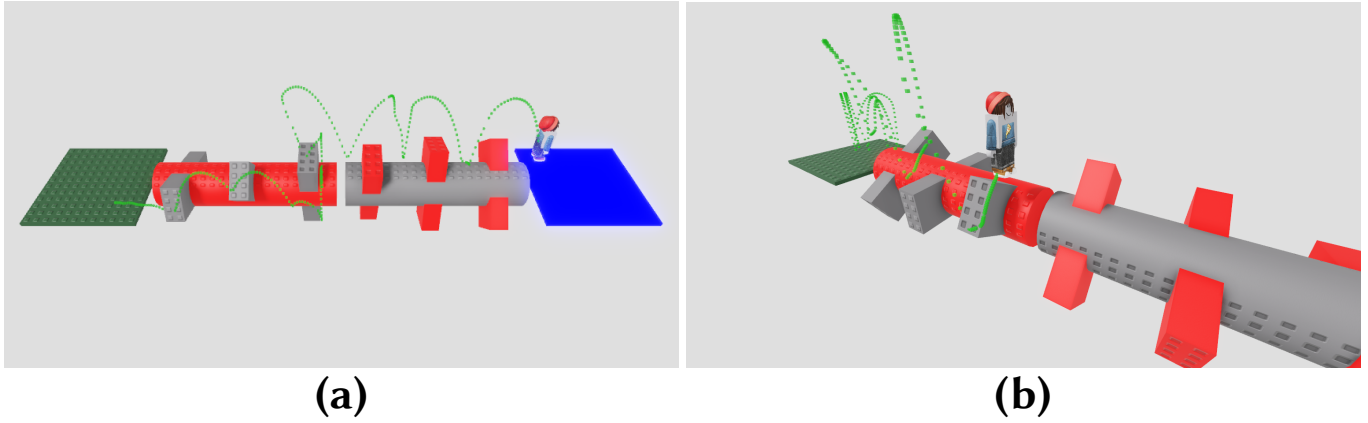


Fig. 9. Feedback from our system supports analysis of an extremely difficult obstacle. **(a)** The obstacle moves too fast to support the designer's intended solution, **(b)** moderate speed allows waiting.

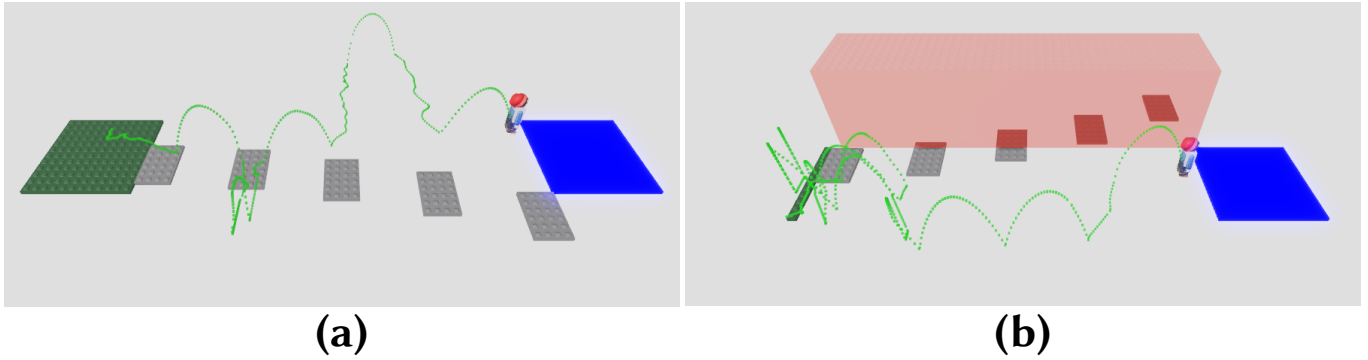


Fig. 10. Our query primitives can express queries that answer questions about complex gameplay. **(a)** The system verifies the first solution the designer envisioned. **(b)** Using our query primitives, the designer verifies that the second, more fluid solution is possible.

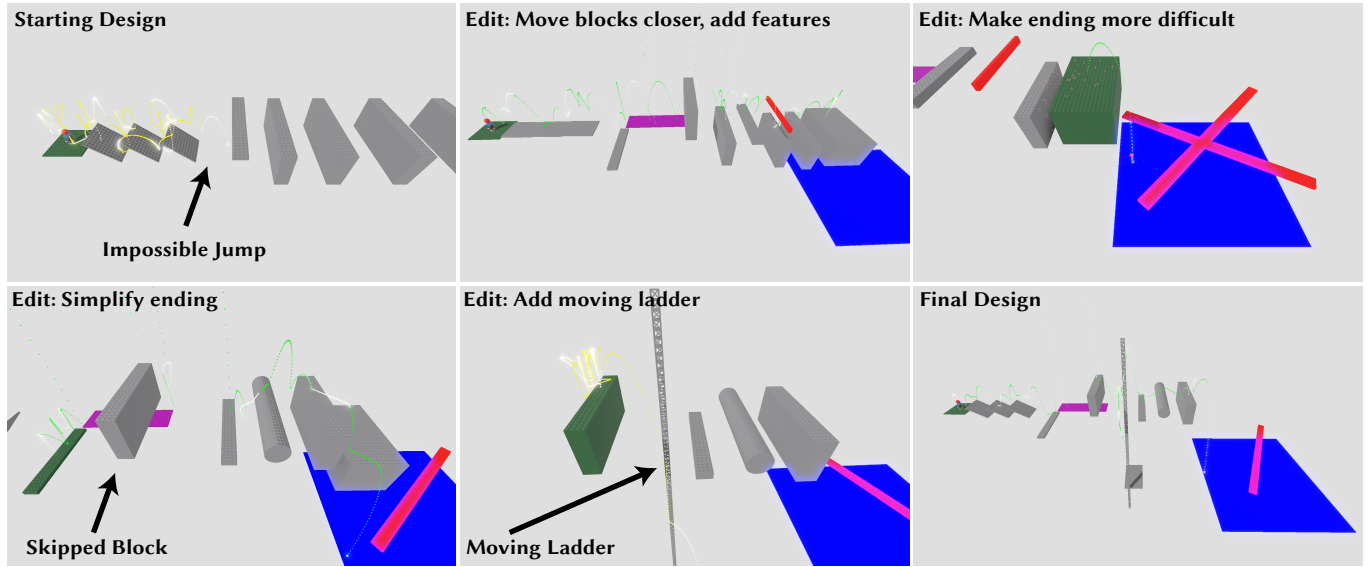


Fig. 11. External Designer 2: The designer used our system to iterate from a draft design with no solution (top left) to a final design with multiple solutions supporting the designer's gameplay goals (bottom right). Each panel shows an edit the designer made in response to feedback from our system. The designer used the system to improve multiple unsolvable designs (top left, bottom center), change their design because a solution was unsatisfying (top right, bottom left), and verify that their final design was solvable in the way they intended (bottom right).

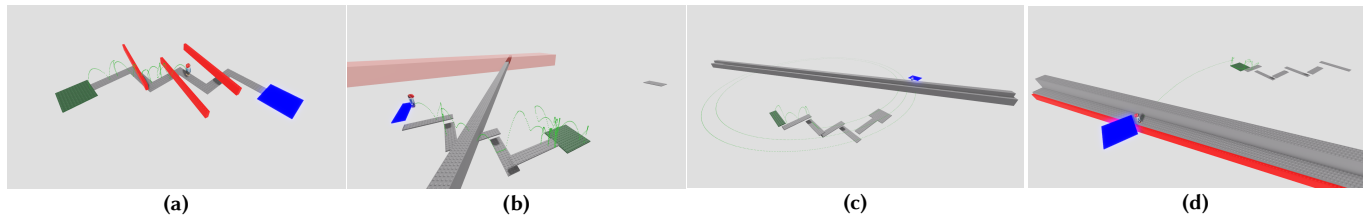


Fig. 12. External Designer 3: This designer used the tool to verify solutions while creating a complex obstacle. (a) Feedback from the system verified that the designers initial design was solvable, but they decided it was too difficult because of the number of jumps over lava spinners required. (b) In the next design iteration, our system verifies that the player can solve the obstacle with a large spinning rod sweeping periodically over the space. (c) On the same obstacle, the designer verifies that, once the player has reached the end of the path, they can catch the spinning rod and be swept to the goal. (d) The designer verifies that an exploit exists that skips walking the path shown in (b).

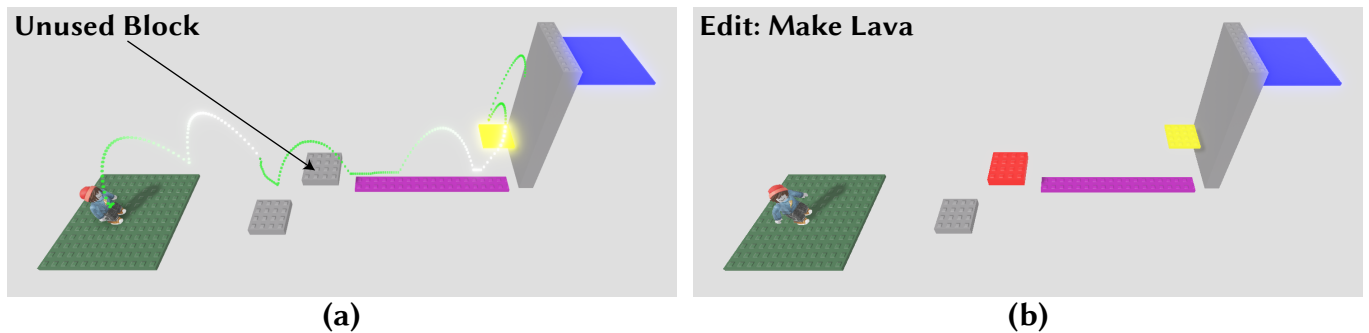


Fig. 13. External Designer 4: This designer used our system to add a level of difficulty to a near final design. After querying for a solution, the designer noticed that the return solution did not touch the second of two moving blocks (a). Upon seeing this solution, the designer realized they could make that block a lava obstacle and maintain solvability (b). This was a design directly inspired by feedback from our system that was not envisioned beforehand by the designer.